RAMDEOBABA UNIVERSITY, NAGPUR

Ramdeo Tekdi, Katol Road, Nagpur – 440 013, Maharashtra, India

SCHOOL / DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

LAB PROJECT REPORT

ON

SMART CITY IoT SENSOR ANALYTICS

USING APACHE CASSANDRA AND APACHE SPARK

Course / Lab: Big Data Analytics Laboratory

Course Code: _____

Submitted By

Sr. No.	Student Name	Roll No.
1	Amaan Khan Pathan	B1-B2-27
2	Ujwal Chikankar	B1-B4-60

Under the Guidance of

Prof. Sonam Chopade

Prof. Samiha Khan

Academic Year: 2026–27

CERTIFICATE

This is to certify that the project entitled “Smart City IoT Sensor Analytics Using Apache Cassandra and Apache Spark” has been successfully completed by Amaan Khan Pathan, Roll No. B1-B2-27, and Ujwal Chikankar, Roll No. B1-B4-60, of the School of Computer Science & Engineering, Ramdeobaba University, Nagpur, as part of the requirements of the laboratory course Big Data Analytics Laboratory during the academic year 2026–27.

The work presented in this report has been carried out under the guidance and supervision of the undersigned.

Project Guide	**Lab In-charge / HOD**
Signature: _____	Signature: _____
Date: _____	Date: _____

DECLARATION

We hereby declare that the project work entitled “Smart City IoT Sensor Analytics Using Apache Cassandra and Apache Spark” is our original academic project work carried out for the laboratory course at Ramdeobaba University, Nagpur. The work has not been submitted elsewhere for the award of any academic qualification.

We further declare that the dataset used in this project is synthetically generated using a fixed random seed and is not presented as real-world city measurement data.

Student Names: Amaan Khan Pathan and Ujwal Chikankar

Roll Nos.: B1-B2-27 and B1-B4-60

Date: _____ Place: Nagpur

ACKNOWLEDGEMENT

We sincerely express our gratitude to Ramdeobaba University, Nagpur, for providing the academic environment and facilities required to complete this laboratory project. We are thankful to our faculty guides, Prof. Sonam Chopade and Prof. Samiha Khan, for their guidance, suggestions, and support throughout the project.

We also thank the School of Computer Science & Engineering, the laboratory staff, and everyone who supported us during the planning, implementation, testing, and documentation of this project.

ABSTRACT

Smart cities use connected sensors to monitor traffic, environmental conditions, parking availability, and waste-bin status. These sensors generate frequent readings that must be stored, validated, processed, and converted into useful information. This project develops a data analytics pipeline using Apache Cassandra for query-oriented storage, Apache Spark with PySpark for data processing, and Streamlit for dashboard presentation. The system uses a synthetically generated dataset containing 11,343 readings from 20 sensors across five locations. The dataset represents four sensor domains: traffic, environment, parking, and waste. A validation layer checks required fields, timestamps, physical value ranges, sensor-specific measurements, duplicate sensor-timestamp pairs, and parking capacity rules. The documented validation outcome is 11,189 accepted readings and 154 rejected readings. Spark processing supports traffic analytics, potential anomaly detection using Z-scores, parking/waste/environment summaries, and sensor-health classification. The project demonstrates a repeatable local pipeline while acknowledging that the dataset is synthetic, Cassandra is deployed as a single local node, and Spark runs locally rather than on a distributed cluster.

Keywords: Smart City, IoT, Apache Cassandra, Apache Spark, PySpark, Streamlit, Data Validation, Z-score.

TABLE OF CONTENTS

1. Introduction .....

2. Problem Statement and Objectives .....

3. Requirements .....

4. Methodology / Algorithm .....

5. System Design .....

6. Implementation .....

7. Results and Discussion .....

8. Conclusion and Future Scope .....

References .....

Appendix .....

1. INTRODUCTION

1.1 Background

A smart city uses connected technologies and sensor systems to observe urban services. Traffic counters, environmental stations, parking systems, and waste-bin sensors can report measurements at regular intervals. These readings can help city operators understand conditions, but only after the data is organised and analysed.

1.2 Motivation

The motivation for this project is to demonstrate how large volumes of sensor readings can be stored and processed using big-data technologies. Instead of examining raw rows manually, the system produces summaries and flags unusual readings for review.

1.3 Problem Definition

The project addresses the challenge of storing and analysing frequent sensor readings from multiple domains while handling missing values, duplicates, impossible measurements, and potentially unusual events.

1.4 Scope

The scope includes synthetic sensor data generation, validation, Cassandra storage, Spark processing, traffic analytics, anomaly detection, parking and waste summaries, environmental summaries, sensor-health monitoring, and dashboard presentation. Real-world sensor ingestion and distributed production deployment are outside the current implementation.

2. PROBLEM STATEMENT AND OBJECTIVES

2.1 Problem Statement

Smart-city sensors generate data continuously across different locations and domains. Traditional manual analysis is not suitable for handling frequent records and identifying data-quality problems. A system is required to store readings efficiently, validate them, calculate useful summaries, and identify potential anomalies.

2.2 Objectives

1. Build a repeatable Apache Cassandra and Apache Spark data pipeline.
2. Validate and clean readings by rejecting missing, duplicate, and impossible records.
3. Analyse traffic, environment, parking, and waste data using PySpark.
4. Detect potential anomalies using a per-sensor Z-score method.
5. Monitor sensors as active, delayed, inactive, or unknown.

2.3 Expected Outcome

The expected outcome is a working local pipeline that ingests synthetic readings, stores valid records in Cassandra, processes bounded data windows with Spark, and publishes summaries and potential anomalies for a Streamlit dashboard. The documented validation result is 11,189 accepted readings out of 11,343.

3. REQUIREMENTS

3.1 Hardware Requirements

- Computer or laptop capable of running Python, Cassandra, and Spark locally.
- Multi-core processor recommended for local data processing.
- Sufficient RAM and disk space for the database, processing environment, and sample CSV.

The project defense material does not specify an exact processor model, RAM size, or storage capacity.

3.2 Software Requirements

Software / Tool	Version / Use
Apache Cassandra	4.1 — data storage
Apache Spark / PySpark	3.5.1 — data processing
Cassandra Spark Connector	3.5.1 — Cassandra-Spark integration
Python	3.10 / 3.11 — implementation
Streamlit	Dashboard presentation
Docker	Reproducible environment

3.3 Dataset / Input Requirements

The input is data/sample_sensor_data.csv, a synthetic dataset generated using seed 42. It contains 11,343 readings, 15 columns, 20 sensors, five locations, and a 15-minute sampling interval. Four sensor types are represented: traffic, environment, parking, and waste.

4. METHODOLOGY / ALGORITHM

4.1 Proposed Method

The proposed method follows a repeatable pipeline: generate or load the synthetic CSV, validate the readings, ingest accepted records into Cassandra, read bounded date windows through the Spark Cassandra Connector, clean and aggregate the data, detect potential anomalies, and publish compact outputs for the dashboard.

4.2 Algorithm

6. Load the synthetic sensor CSV generated with seed 42.
7. Validate required fields, timestamps, value ranges, duplicates, and parking capacity.
8. Reject invalid records and log the rejection reason.
9. Insert valid records into query-oriented Cassandra tables using upserts.
10. Read bounded date windows from Cassandra using PySpark.
11. Calculate traffic, environmental, parking, and waste summaries.
12. Calculate per-sensor Z-scores and flag readings where $|Z| \geq 3$.
13. Classify sensor health and publish results for dashboard display.

4.3 Flowchart

Dataset CSV → Validation and Cleaning → Cassandra Storage → Spark Processing → Analytics and Z-score Detection → Output Files → Streamlit Dashboard

4.4 Pseudocode

BEGIN
Load sensor dataset
For each reading: validate required fields, timestamp, ranges, duplicates, and capacity
If valid: store in Cassandra; otherwise: record rejection reason
Read bounded data window with Spark
Clean and aggregate readings
Detect potential anomalies using Z-score
Update sensor health and publish results
Display results on dashboard
END

5. SYSTEM DESIGN

5.1 System Architecture

Input layer: synthetic CSV with 11,343 sensor readings.
Validation layer: required-field, timestamp, range, duplicate, and capacity checks.
Storage layer: Apache Cassandra 4.1, keyspace smart_city, four query-oriented tables.
Processing layer: PySpark 3.5.1 with Cassandra connector, DataFrames, aggregations, windows, caching, and Z-score detection.
Presentation layer: Streamlit dashboard.

5.2 Modules

- Data generation and ingestion — creates repeatable sensor records and loads them.
- Validation module — checks data quality and logs rejected rows.
- Cassandra storage module — stores readings and status/anomaly records.
- Spark analytics module — calculates domain summaries and potential anomalies.
- Sensor health module — classifies status using last-reading age.
- Dashboard module — presents processed outputs for inspection.

5.3 Data Flow / Workflow

Data moves from the synthetic CSV to validation. Accepted readings are stored in Cassandra. Spark reads bounded date windows, performs cleaning and analytics, and publishes result files. Streamlit displays the resulting information. Cassandra tables are organised around query patterns to avoid unrestricted table scans.

6. IMPLEMENTATION

6.1 Development Environment

The documented environment uses Python 3.10/3.11, Apache Cassandra 4.1, Apache Spark/PySpark 3.5.1, the Cassandra Spark Connector 3.5.1, Docker, and Streamlit. Cassandra is deployed as a single local node with replication factor 1, while Spark runs locally.

6.2 Module-wise Implementation

Data validation

The validation stage checks required columns and UTC timestamps, valid physical ranges, sensor-specific required values, duplicate sensor-timestamp pairs, and parking capacity. Each rejected row is associated with a reason.

Cassandra storage

The smart_city keyspace contains sensor_readings_by_location, sensor_readings_by_type, latest_sensor_status, and detected_anomalies. Primary-key upserts support safe repeat ingestion.

Spark processing

Spark reads bounded windows, cleans records, uses DataFrame groupBy/aggregation and window functions, caches reusable results, and calculates Z-scores.

Dashboard publishing

Processed CSV results and potential anomaly records are published for display in the Streamlit dashboard.

6.3 Important Code Snippets

The supplied defense deck documents the pipeline design and processing concepts but does not include complete source-code listings. Therefore, full code is not reproduced here. The key documented formula is the standard score:

$$Z = (x - \text{mean}) / \text{standard deviation}$$

7. RESULTS AND DISCUSSION

7.1 Experimental Setup

The validation experiment uses data/sample_sensor_data.csv containing 11,343 synthetic readings. The generated sample represents 20 sensors at five locations, with one reading every 15 minutes. The validation rules are applied before records are stored.

7.2 Results

Validation outcome	Count	Share
Total input readings	11,343	100%
Accepted readings	11,189	98.6%
Rejected readings	154	1.4%
Missing required value	64	—
Duplicate sensor-timestamp	51	—
Impossible value	39	—

Accepted readings by domain: environment 2,982; traffic 2,980; waste 2,614; parking 2,613.

7.3 Performance Analysis

The validation pass rate is 98.6%. The 154 rejected records represent 1.4% of the input. The available project material reports validation counts but does not provide throughput, latency, or comparative benchmark measurements. Therefore, no performance superiority claim is made.

7.4 Discussion

The results show that the validation layer detects missing required values, duplicate sensor-timestamp records, and impossible values before storage. The analytics pipeline is designed to support traffic peaks, parking and waste status, environmental summaries, potential anomalies, and sensor health. Anomaly flags require human review.

8. CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

The project demonstrates a repeatable pipeline for transforming synthetic smart-city sensor readings into useful analytics. Apache Cassandra stores the data using query-oriented tables, PySpark validates and processes bounded data windows, and Streamlit presents the results. The project covers traffic, environment, parking, waste, potential anomaly detection, and sensor-health monitoring. Out of 11,343 input readings, 11,189 passed validation and 154 were rejected. The project is a local demonstration and does not claim real-world deployment or distributed performance.

8.2 Future Scope

- Integrate live sensor ingestion using Kafka or MQTT.
- Use Spark Structured Streaming for continuous processing.
- Evaluate Isolation Forest alongside Z-score anomaly detection.
- Deploy multi-node Cassandra using NetworkTopologyStrategy.
- Conduct fair benchmarks on larger datasets.
- Extend scope to PM2.5/PM10 measurements and waste-bin time-to-full forecasting.

REFERENCES

[1] Apache Software Foundation, “Apache Cassandra Documentation,” https://cassandra.apache.org/doc/.

[2] Apache Software Foundation, “Apache Spark Documentation,” https://spark.apache.org/docs/latest/.

[3] DataStax, “Spark Cassandra Connector,” https://github.com/datastax/spark-cassandra-connector.

[4] Streamlit, “Streamlit Documentation,” https://docs.streamlit.io/.

[5] Standard statistical Z-score method for identifying observations that differ from a group mean.

APPENDIX

Appendix A – Source Code

The project defense material identifies the technologies, pipeline stages, Cassandra tables, validation rules, and Spark processing concepts. It does not include the complete source-code listing. Add the final implementation files or repository details here when available.

Appendix B – Dataset

Dataset file: data/sample_sensor_data.csv
Type: synthetic sensor dataset
Generation seed: 42
Rows: 11,343
Sensors: 20
Locations: 5
Sampling interval: 15 minutes
Columns: 15
Domains: traffic, environment, parking, waste.

Appendix C – Additional Screenshots

Insert screenshots of the Streamlit dashboard, Cassandra tables, Spark processing output, validation summary, and anomaly results in this section. The uploaded presentation documents the result counts but does not provide separate screenshot files in the material used to prepare this report.